

The while loop, in practice

One step too many, one too few · TC1037 · Lecture 06

Dr. Leopoldo Rodríguez Salazar

Friday 9 October 2026

Tecnológico de Monterrey · Campus Querétaro · Group 510

Five things, one evening

reply to an email	5 min
Homework 4	90 min
buy groceries	30 min
call home	15 min
read for Tuesday	40 min

Which order?

Algorithms to Live By, ch. 5 — Scheduling

Shortest Processing Time first (Smith, 1954)

Shortest first: 5, 15, 30, 40, 90

done at 5, 20, 50, 90, 180

total waiting **345**

Longest first: 90, 40, 30, 15, 5

done at 90, 130, 160, 175, 180

total waiting **735**

If the goal is the **smallest total wait**, shortest-first is the only right order.

A long task at the front makes *everything behind it* wait.

And how did you do it? **while** tasks remain: pick the shortest, do it, cross it off. A loop — with a list that shrinks every turn.

Your turn — 3 minutes, on paper

Shortest Processing Time, as an algorithm.

INPUT: a list of tasks with durations OUTPUT: the order

The loop, the exit, and the one step that *chooses*.

Keep the paper

Week 7 you write it in Python, with a list.

Every loop today has the same shape: something left to do, a step that reduces it, an exit you can name.

Exercise 1 — `countdown(n)` · the test

Spec: display n , $n-1$, ..., 1, one per line, then Liftoff!

```
1 def countdown(n):  
2     while n > 0:  
3         print(n)  
4         n = n - 1  
5     print("Liftoff!")
```

Shell: `countdown(3)` · `countdown(1)` ·
`countdown(0)`

Change `>` to `>=`: a 0 appears. The spec said
down to 1. One character, one extra turn.

Exercise 2 — digits(n) · the start

```
def digits(n):  
    count = 1  
    while n >= 10:  
        n = n // 10  
        count = count + 1  
    return count
```

`n // 10` drops the last digit: `1234 // 10` is `123`.

✗ The obvious version

`count = 0`, while `n > 0`
gives `digits(0) → 0`.

✓ Every number has at least one digit

Start at 1; count the *extra* ones.

Where the counter starts is a decision
about the smallest input.

Shell: `digits(1000) 4 · digits(999) 3 · digits(10) 2 · digits(7) 1 · digits(0) 1`

Exercise 3 — `play(secret)` · the step order, and *random*

```
import random

def play(secret):
    attempts = 0
    guess = int(input("Guess: "))
    attempts = attempts + 1
    while guess != secret:
        if guess < secret:
            print("Higher.")
        else:
            print("Lower.")
        guess = int(input("Guess: "))
        attempts = attempts + 1
    return attempts
```

`main()` picks the secret:

`random.randint(1, 100)` — both ends included.

The sentinel idiom again — the “sentinel” is the right answer.

The counter goes up right after each `input()`. Anywhere else: off by one.

Shell: `play(50)`, type 50 → 1.

Type 25, 75, 50 → 3.

A function you can test keeps its randomness in `main`.

Exercise 4 — $\text{gcd}(a, b)$ · termination

```
def gcd(a, b):  
    while a != b:  
        if a > b:  
            a = a - b  
        else:  
            b = b - a  
    return a
```

Euclid, 300 BC. $\text{gcd}(48, 18)$: 30, 12, 6, 6 $\rightarrow$ 6.

✓ Why it stops

Every turn one number gets *strictly smaller*; neither can go below 1. So $a + b$ falls every turn and has a floor. **Name the quantity, name the floor.**

✗ $\text{gcd}(5, 0)$

$a = 5 - 0 = 5$, forever. The docstring said *positive*. The caller lied — and in main, *you* are the caller.

The gallery — each one step from right. Predict, then name the place.

(a) “the numbers from 1 to 5”

```
i = 1
while i < 5:
    print(i)
    i = i + 1
```

(b) “n times”, $n = 3$

```
k = 0
while k <= 3:
    print("b")
    k = k + 1
```

(c) “the sum of 1..n”, $n = 4$

```
total = 0
j = 1
while j <= n:
    j = j + 1
    total = total + j
```

(d) “read until -1, count the readings”

```
count = 0
x = int(input())
while x != -1:
    x = int(input())
    count = count + 1
```

Three places: **test** · **start** · **step order**. Which is each?

Write the four · from 18:28 · algorithm on paper first

File	Function	Shell tests to write down
A_Matricula_w1.py	countdown(n)	countdown(3), (1), (0)
A_Matricula_w2.py	digits(n)	(0) 1 · (7) 1 · (10) 2 · (1000) 4
A_Matricula_w3.py	play(secret); main: randint	play(50): 50 → 1; 25 75 50 → 3
A_Matricula_w4.py	gcd(a, b); main refuses ≤ 0	(48,18) 6 · (1071,462) 21 · (7,7) 7

Each: one function + main, exam format. **Shell tests before main().**

Photos of the algorithms + the four files → **Week 3 evidence box** (Canvas). Not graded; your record; I read it.

Finished? Make play tell the user how far from seven they were.

1. **Homework 4** — `collatz_steps`, `reverse_digits`, `main`; a *why it stops* sentence per loop. **Sunday 11 Oct, 23:59.**
2. **Lab 3 files** — C1, C2 + photo → *Laboratory 3*. Same Sunday.
3. *Self-study: counter-controlled repetition* (Canvas, Week 4) — ten minutes, so `for` is a word you have seen.
4. The levelling module is closed. The course moves on.

Quick exam 3: Tuesday 13 October, first 20 minutes.

The `while` loop — the gallery, `read · predict · fix`, plus one loop to write as a function.

Test the edges, not the middle.